Ruta paso a paso del Livecoding

Secuencia cronologica con libreto y codigo en cada paso

■ La secuencia completa (memorizala como mapa)

0. Escuchar instrucciones y preguntar → **1.** Presentar plantilla (1 min) → **2.** Ajustar config → **3.** Crear Page Objects (con codegen) → **4.** Registrar en Fixtures → **5.** Crear Specs (AAA) → **6.** Ejecutar en vivo y narrar → **7.** Ultimo escenario con IA + MCP → **8.** Reporte Allure y cierre.

PASO 0 Escuchar instrucciones y hacer 3 preguntas min 0-5

Al entrar a la reunion te daran la web y los casos. Anota la URL y los flujos. Antes de tocar codigo, haz maximo 3 preguntas:

■ Libreto

"Perfecto. Tres preguntas rápidas antes de empezar: ¿tienen credenciales de prueba que deba usar? ¿Hay algún flujo que quieran priorizar? ¿Prefieren que lo corra en un solo navegador o cross-browser?"

✓ Mientras te responden

Abre la URL en tu navegador normal y dale un vistazo de 30 segundos al flujo que te pidieron. Ya estas explorando sin gastar tiempo.

PASO 1 Presentar la plantilla (breve, sin extenderse) min 5-7

Comparte pantalla con WebStorm abierto y recita esto casi literal. No entres en detalle de cada archivo; eso lo explicas cuando lo uses.

■ Libreto

"Esta es una plantilla que ya tengo predefinida para proyectos Playwright con TypeScript. Con el comando 'npm init playwright' se genera la base y la carpeta e2e, y yo, siguiendo el esquema gTAA de ISTQB, anexé tres carpetas adicionales: fixtures, para instanciar los pages en una sola clase; pages, que son los Page Object Model; specs, que son los tests; y utils, que son helpers reutilizables. También dejé integrado Allure para la reportería. Con esto no gasto tiempo en configuración y voy directo a automatizar."

```
# (solo mencionar, no ejecutar - la plantilla ya existe)
npm init playwright@latest      # genero base + carpeta e2e
```

PASO 2 Ajustar playwright.config.ts min 7-10

Abre el config y cambia SOLO dos cosas mientras narras:

```
use: {
  baseURL: 'https://LA-URL-QUE-TE-DIERON',   // 1) la app del ejercicio
  testIdAttribute: 'data-test',              // 2) ajustar, o ignorar si no hay data-*
  trace: 'retain-on-failure',
  screenshot: 'only-on-failure',
},
```

■ Libreto

"Primero apunto el baseURL a la aplicación del ejercicio: esto me permite navegar con rutas relativas y, si mañana cambia el ambiente, toco una sola línea. Segundo, reviso si el sitio expone atributos de test tipo data-test; lo verifico inspeccionando un elemento. Si los tiene, configuro testIdAttribute; si no, trabajaré con locators semánticos como getByRole y no necesito este parámetro."

✓ Verificacion rapida del atributo (30 seg)

Click derecho sobre un input importante → Inspeccionar → mira sus atributos. Si ves `data-test="..."` o `data-testid="..."`, configura ese valor. Si solo hay id/name con significado (ej. `id="email"`), iras por `locator('#email')` y roles.

PASO 3 — Crear los Page Objects (usando codegen) — min 10-25

3a. Descubrir locators con codegen

Antes de escribir el POM, abre codegen para explorar la app y ver que locators sugiere Playwright:

```
npx playwright codegen https://LA-URL-QUE-TE-DIERON
# OJO: npx (con n), y la URL va sin &lt; &gt; ni comillas
```

■ Libreto

"Voy a usar codegen, el grabador de Playwright: al interactuar con la página me sugiere el locator óptimo para cada elemento. Si me sugiere getByRole o getByLabel, el sitio tiene buena semántica accesible; si cae a CSS, ajusto la estrategia. Esto me sirve de reconocimiento y de borrador al mismo tiempo. Los locators que me sugiera no los copio ciegamente: los reviso y los organizo dentro de mi Page Object."

3b. Crear el archivo del POM y explicar los imports

Crea `e2e/pages/LoginPage.ts` (o copia tu plantilla SamplePage). Mientras escribes los imports, narra que es cada uno:

```
import { type Page, type Locator, expect } from '@playwright/test';
```

■ Libreto

"Importo tres cosas del paquete de Playwright: el tipo Page, que representa la pestaña del navegador; el tipo Locator, que es un puntero a un elemento del DOM; y expect, la función de aserciones. Page y Locator llevan la palabra 'type' porque solo los uso para tipar, no se ejecutan; expect sí se ejecuta, por eso va sin 'type'."

3c. La clase, los locators y el constructor

```
export class LoginPage {
  readonly page: Page;
  private readonly email: Locator;
  private readonly password: Locator;
  private readonly submitButton: Locator;
  private readonly welcomeMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.email = page.locator('#email');          // ancla estable (id con significado)
    this.password = page.locator('#password');
    this.submitButton = page.getByRole('button', { name: 'Login' });
    this.welcomeMessage = page.getByText('Bienvenido');   // ajustar al real
  }
```

■ Libreto

"Declaro los locators como private readonly: private porque son detalle interno de la página, el test no debe tocarlos; readonly porque una vez asignados no cambian. El constructor recibe page porque los locators necesitan saber sobre qué pestaña operar: no la creo yo aquí, me la inyecta la fixture. Para las anclas priorizo getByRole por semántica, y uso id o name solo cuando son estables y con significado, como #email. Nunca ids autogenerados ni clases con hash."

3d. Los metodos de accion (goto, login)

```
  async goto(): Promise&lt;void&gt; {
    await this.page.goto('/');            // '/' se resuelve contra el baseURL
  }

  async login(email: string, pass: string): Promise&lt;void&gt; {
    await this.email.fill(email);
    await this.password.fill(pass);
    await this.submitButton.click();
  }
```

■ Libreto

"Los métodos son los servicios que ofrece la página. goto navega a la raíz: el slash se combina con el baseURL del config. login recibe las credenciales como parámetros tipados string, llena el formulario con fill y hace click. Son async porque las operaciones de navegador tardan, y el await pausa hasta que cada una termine. El test nunca sabrá que login son tres pasos: solo pide el servicio."

3e. La asercion (validar login exitoso)

```
  async expectLoginSuccessful(): Promise&lt;void&gt; {
    await expect(this.welcomeMessage).toBeVisible();
    // o por URL:  await expect(this.page).toHaveURL(/.*dashboard/);
  }
}
```

■ Libreto

"La validación vive dentro del Page Object: verifico que tras el login aparece el mensaje o elemento que confirma el éxito. Uso web-first assertions: toBeVisible reintenta automáticamente hasta que el elemento aparezca o venza el timeout, así no necesito esperas fijas. Esto es lo que elimina los tests flaky."

PASO 4 Registrar el POM en las Fixtures min 25-30

Abre `e2e/fixtures/test.ts` y registra el page creado. Narra la semantica de cada linea:

```
import { test as base, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

type Pages = {
  loginPage: LoginPage;
};

export const test = base.extend&lt;Pages&gt;({
  loginPage: async ({ page }, use) =&gt; {
    await use(new LoginPage(page));
  },
});

export { expect };
```

■ Libreto

"Aquí importo el test original de Playwright pero lo renombro a 'base' con la palabra 'as', porque voy a construir mi propia versión extendida. El type Pages declara qué fixtures nuevas existen y de qué tipo es cada una: eso me da autocompletado en el IDE. Luego base.extend crea mi test ampliado: cada fixture es una función que recibe la página y una función use. Dentro hago new LoginPage(page), que construye el Page Object inyectándole la pestaña, y await use lo entrega al test. Todo lo que pusiera antes del use sería setup, y lo que pusiera después sería teardown. Finalmente re-exporto expect para que los specs importen todo desde un solo archivo. El resultado: los tests piden loginPage entre llaves y lo reciben ya construido, sin escribir new en ningún spec."

■ Si creas mas POMs (ej. DashboardPage)

Repites el patron: 1) import de la clase, 2) linea en type Pages, 3) bloque en base.extend. Tres toques por cada page nuevo.

PASO 5 Crear los tests en specs (patron AAA) min 30-50

5a. Explicar AAA en una frase

■ **Libreto**

"Estructuro cada test con el patrón Arrange-Act-Assert: Arrange prepara el estado (navegar, precondiciones), Act ejecuta la acción que quiero probar, y Assert verifica el resultado esperado. Tres bloques claros que hacen el test legible para cualquiera."

5b. Los imports del spec

```
import { test } from '../fixtures/test';
import { assertWithEvidence } from '../utils/evidence';
```

■ **Libreto**

"Fíjense que importo test desde MIS fixtures, no desde @playwright/test: así recibo los page objects ya instanciados. Y traigo mi helper de evidencia, que adjunta un screenshot al reporte cada vez que una validación pasa. No importo expect aquí porque las aserciones viven dentro de los page objects; si lo importara sin usarlo sería un import muerto."

5c. Construir el test: describe + test + fixtures

```
test.describe('Autenticacion', () =&gt; {

  test('Login exitoso con credenciales validas @Smoke @Auth', async ({
    page, loginPage,
  }) =&gt; {
    // Arrange: preparar
    await loginPage.goto();

    // Act: la accion bajo prueba
    await loginPage.login('usuario@correo.com', 'password123');

    // Assert: verificar con evidencia
    await assertWithEvidence(page, 'Login exitoso - elemento de bienvenida visible', async () =&gt; {
      await loginPage.expectLoginSuccessful();
    });
  });
});
```

■ **Libreto**

"test.describe agrupa tests relacionados bajo un título: sirve para organizar el reporte y aplicar hooks comunes como beforeEach. Recibe el nombre del grupo y una función flecha que contiene los tests. Dentro, cada test recibe su nombre —donde incluyo tags como @Smoke para poder filtrar la ejecución— y una función async que desestructura entre llaves las fixtures que necesita: aquí pido page y loginPage, y llegan listos gracias a la fixture. El cuerpo sigue AAA: goto es el Arrange, login es el Act, y el Assert va envuelto en mi helper para que deje screenshot en Allure."

✓ **Si piden un caso negativo o data-driven**

Negativo: mismo esqueleto, credenciales invalidas y `expectError('mensaje')` en el POM. Data-driven: un array de casos y `for (const c of casos) { test(...) }` - un test por cada juego de datos, equivalente al Scenario Outline de Gherkin.

PASO 6 Ejecutar en vivo y narrar la pantalla min 50-60

```
npm run test:ui        # modo UI: time-travel visual (recomendado para demo)
# alternativas:
npm run test:headed    # navegador visible, ejecucion normal
npm test               # headless: sin ventana, como corre en CI
```

■ Libreto

"Voy a ejecutar en modo UI, que es la herramienta de observación de Playwright: a la izquierda ven los tests; al correr uno, en el centro se ve cada acción sobre la app y puedo navegar la línea de tiempo hacia atrás y adelante, viendo el estado del DOM en cada paso. Vean cómo llena el formulario, hace click y la aserción espera al elemento de bienvenida... y pasa en verde. Aclaro: esta ventana es para desarrollo y demo; la ejecución real y la de CI corre headless con npm test, que es más rápida."

■ Si un test falla en vivo (no entres en panico)

Di: "Perfecto, esto es justo lo que el framework resuelve: el reporte me da el screenshot del fallo y el trace navegable". Abre el error, revisa el locator, corrige y re-ejecuta. Manejar un fallo con calma suma MAS puntos que no fallar.

PASO 7

Ultimo escenario con IA: Claude + Playwright MCP

min 60-80

7a. Introducir el MCP (proposito + cuando si / cuando no)

■ **Libreto**

"Para el siguiente escenario voy a usar un flujo asistido por IA con el MCP de Playwright. El MCP es un protocolo que conecta a Claude con un navegador real: Claude navega la aplicación, lee el árbol de accesibilidad y genera código con locators verificados contra el DOM, no inventados. Lo uso porque acelera la autoría cuando hay flujos con muchas páginas cuyos locators no conozco. Ahora, es importante el criterio: el MCP sirve para EXPLORAR y GENERAR BORRADORES; no lo uso para ejecutar la suite en CI —eso lo hace el runner con npm test, que es determinista— ni acepto su código sin revisarlo, porque puede alucinar locators o meter malas prácticas. La IA es copiloto de autoría; la arquitectura y la aprobación final son mías."

7b. Mencionar el comando de instalacion (sin ejecutarlo)

```
# Ya lo tengo instalado; se registro con este comando (requiere Node 18+):
claude mcp add playwright npx @playwright/mcp@latest
```

■ **Libreto**

"El MCP ya lo tengo registrado en el proyecto; se instala con 'claude mcp add playwright' apuntando al paquete oficial de Microsoft. Solo lo menciono, no hace falta repetirlo."

7c. El prompt estandar (copiar, pegar y adaptar en Claude Code)

✦ **PROMPT - primero el contexto**

Lee estos archivos para entender la arquitectura del proyecto antes de generar nada:
- e2e/fixtures/test.ts
- e2e/pages/LoginPage.ts (el POM ya creado)
- e2e/specs/login.spec.ts (el spec ya creado)
- e2e/utils/evidence.ts

✦ **PROMPT - luego la generacion (adaptar lo subrayado)**

Usa **Playwright MCP** para abrir <URL-de-la-app>, hacer login con <credenciales> y explorar el flujo de <describir el escenario que pidieron, ej.: crear un registro y validar que aparece en la lista>. Identifica los locators reales mediante el arbol de accesibilidad.

Luego genera, siguiendo EXACTAMENTE el estilo de mis archivos existentes:
1. Un Page Object en e2e/pages/<Nombre>Page.ts: locators private readonly, priorizando getByRole/getByLabel y usando id/name estables solo si es necesario; con las aserciones (expectXxx) DENTRO del POM.
2. El registro de la fixture en e2e/fixtures/test.ts (import + type Pages + extend).
3. Un spec en e2e/specs/<nombre>.spec.ts con patron Arrange-Act-Assert, tags @Smoke @<Modulo> y assertWithEvidence en las validaciones.

Reglas estrictas: NO uses waitForTimeout. NO uses XPath posicional. NO uses ids autogenerados ni clases con hash. Muestrame los cambios ANTES de escribir los archivos.

7d. Revisar el diff y aprobar (narrar la revision)

■ **Libreto**

"Claude me muestra los archivos que propone antes de escribirlos. Reviso contra mis estándares: que los locators sean semánticos o anclas estables, que las aserciones estén dentro del page object, que haya registrado la fixture, que no haya esperas fijas ni imports muertos. Todo correcto: apruebo la escritura. Este paso de revisión es innegociable: la IA propone, yo dispongo."

7e. Ejecutar el escenario generado

```
npx playwright test e2e/specs/NOMBRE.spec.ts --headed
```

■ **Libreto**

"Ejecuto solo el spec nuevo en modo visible para que veamos el flujo que la IA construyó bajo nuestro patrón... y pasa en verde. Nótese que el resultado es indistinguible de un test escrito a mano: misma arquitectura, mismos estándares. La diferencia fue el tiempo de autoría."

PASO 8 Reporte Allure y cierre min 80-90

```
npm run allure:serve
```

■ **Libreto**

"Cierro levantando el reporte Allure: aquí se ven las suites agrupadas, cada test con sus pasos nombrados y el screenshot de cada validación que pasó; si algo hubiera fallado, tendríamos la captura del fallo y el trace navegable. Este mismo reporte es el que publicaría el pipeline de CI. Como siguientes pasos naturales quedarían: integrar la suite a GitHub Actions, ampliar cobertura con casos negativos y data-driven, y parametrizar ambientes por variable de entorno. ¿Preguntas?"

Regla de oro: llega a VERDE con el primer test lo antes posible (min 30-35). Un test corriendo temprano te da calma y margen. Piensa en voz alta SIEMPRE: el libreto no se recita, se usa como ancla. El codigo es la mitad de la nota; tu razonamiento es la otra mitad.